

La Arquitectura del Gigante

Sesión 4 — Diseño lógico de escalabilidad (catálogo, usuarios, interacciones)

Alumno(a)

Proyecto / Negocio

Javier Rodrigo Calcin Paja

Tienda Virtual "CP Cards & Pokémon"

Ejecuta primero [demo-escalabilidad.html](#) con tu propio catálogo (o el de ejemplo) para obtener números reales de tu navegador, y complétalos aquí.

1. Cuello de botella actual vs. solución escalable

ELEMENTO	CÓMO FUNCIONA HOY (CUELLO DE BOTELLA)	CÓMO LO RESOLVERÍA UN MARKETPLACE GRANDE
EJEMPLO Catálogo	Cada producto está escrito a mano en el HTML de la categoría.	Los productos son datos; se indexan y se muestran paginados (10-20 por página).
Catálogo	Cada carta está escrita de forma estática en el HTML o se renderizan cientos de cartas juntas en el DOM al cargar la página.	Los datos se almacenan en MySQL; se consultan de forma asíncrona mediante un endpoint paginado (ej. 12-24 cartas por página) con <i>Lazy Loading</i> .
Usuarios / Clientes	Gestión manual por mensajería sin registros estructurados de autenticación o historial centralizado.	Base de datos relacional con tabla Usuarios, autenticación mediante tokens JWT y contraseñas ecryptadas con bcrypt.
Búsqueda y Filtros	Búsqueda manual por chat o mediante filtros simples en JavaScript recorriendo todos los elementos en el cliente.	Consultas SQL optimizadas con LIKE e índices en campos clave (nombre_carta, expansion, rareza).
Pedidos	Confirmación manual vía capturas de pantalla enviadas por WhatsApp y revisión en cuenta bancaria personal.	Flujo transaccional automatizado en Backend con pasarela de pagos integrada, gestión de estados de pedido y decremento de stock en BD.
Interacciones (Chat/Soporte)	Mensajes directos en tiempo real por WhatsApp donde el tiempo de respuesta cae si hay múltiples consultas simultáneas.	Bot interactivo de primera línea para consultar stock y preguntas frecuentes, derivando a un sistema de tickets o WhatsApp solo para casos complejos.

+ Agregar fila

Filas sugeridas: Catálogo, Usuarios/clientes, Búsqueda, Pedidos, Interacciones (chat/soporte).

2. Resultados del benchmark (demo-escalabilidad.html)

Tamaño de catálogo probado

1,000 cartas Pokémon (objetos JSON con precio USD, expansión y rareza)

Tiempo del patrón actual (renderizar todo)

145 ms (Generación masiva de elementos DOM, alto consumo de memoria del navegador)

Tiempo del patrón escalable (índice + paginación)

12 ms (Indexación en memoria y renderizado de solo 12 elementos por página)

3. Diseño lógico propuesto

Explica, en 5-8 líneas, cómo debería organizarse la lógica interna de tu plataforma (catálogo, usuarios, interacciones) para poder crecer sin colapsar, apoyándote en el DER de la Sesión 3.

Producto y Detalle_Pedido descritas en el DER, realizando consultas paginadas con cláusulas LIMIT y OFFSET en SQL para enviar al cliente solo los datos necesarios. Capa de Usuarios: Se gestionará a través de la tabla Cliente, manteniendo sesiones seguras para evitar consultas innecesarias de perfil en cada acción. Capa de Interacciones y Pedidos: Las compras se ejecutarán dentro de transacciones SQL (BEGIN ... COMMIT) que aseguren la consistencia de la tabla Pedido y actualicen el inventario de la tabla Producto en milisegundos. Esta separación de

4. Conclusión

La prueba demuestra que la escalabilidad de una plataforma E-commerce no depende únicamente de la potencia del servidor, sino de la eficiencia con la que se procesan y entregan los datos. Renderizar un catálogo masivo de golpe degrada drásticamente la experiencia del usuario y consume recursos innecesarios; implementar un diseño lógico basado en indexación, paginación y la estructura relacional de la base de datos es la única vía para escalar el negocio a